

Experiment No. 10

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory

What is hyperledger?

Hyperledger Fabric is an open-source, enterprise-grade, permissioned distributed ledger framework hosted by the Linux Foundation. Designed primarily for business applications, it distinguishes itself through a modular architecture that allows for customizable consensus mechanisms, identity management, and smart contracts (known as chaincode) written in general-purpose languages like Go and Java.

Key characteristics include:

- **Privacy and Confidentiality:** Unlike public blockchains, Fabric operates on a permissioned network where participants are known and authenticated, utilizing channels to partition data so that only authorized members can access specific transactions.
- **Scalability and Performance:** It separates transaction ordering from validation, enabling high throughput and efficient scaling for complex enterprise use cases in industries such as supply chain, finance, and healthcare.
- **Modularity:** The platform uses plug-and-play components, allowing organizations to tailor the system to their specific trust assumptions and regulatory requirements

Architecture of Hyperledger

1. Consensus Layer

- Creates an agreement on the ordering and confirms the accuracy of the transaction database that comprises a block.
- Confirms that all transactions in a proposed block are correct according to approval and consensus policies.
- Interfaces with the smart-contract layer and relies on it to validate the accuracy of an ordered transaction database in a block.
- They're also widely utilized to synchronize data across a decentralized network and assure transaction consistency and transparency.

2. Smart Contract Layer:

- The smart contract layer verifies each transaction by guaranteeing that it adheres to the transaction's policy and contract and invalid transactions are denied and may be removed from consideration for inclusion in a block.
- Smart contracts are classified into two types:
 - **installed smart contracts-** Before the network is launched, installed smart contracts implement business logic on the validators.
 - **On-chain smart contracts-** On-chain smart contracts implement business rules in the form of a transaction that is committed to the blockchain and then invoked by subsequent transactions. The code that describes the business logic forms part of the ledger with on-chain smart contracts.
- In Hyperledger Fabric, a smart contract is a program known as chaincode. Chaincode can be developed in Go, JavaScript (node.js), and, in the future, additional programming languages such as Java that define a predefined interface. Chaincode is run in a secure Docker container that is separate from the endorsing peer process.

3. Communication Layer:

- Communication Layer is in charge of peer-to-peer message transfer between nodes in a shared ledger instance.
- The communication layer is used by the consensus layer to communicate with the client and other network peers.
- The algorithm must function exactly like a single node system, executing each transaction atomically one at a time.
- If communication does not fail, then each non-faulty node will finally get every submitted transaction.
- TLS is used for secure communication among nodes in the Fabric. TLS communication can employ both one-way (server only) and 2 different (server and client) authentication.

4. Identity Service:

- Allows for the formation of a trusted root during the configuration of a blockchain instance, the enrolment and registration of identities or systems entities during network operation, and the administration of changes such as drops, additions, and revocations. It also offers authentication and permission.
- The smart contract layer employs the identity services layer to authenticate and approve the entity requesting to execute the smart contract while processing the transaction.
- Hyperledger Fabric provides a personal identity service that handles user IDs and authenticates all network participants to enable permissioned networks. Access control lists can be utilized to add layers of permission by authorizing certain network actions.
- Certification Authorities are in charge of managing certificates (or CA). Fabric CA is Hyperledger's Certification Authority

5. Policy Service:

- Policy Services is in charge of policy management for the system's numerous policies, including the endorsement policy, consensus policy, and group management policy. It communicates with and is dependent on another module to enforce the different policies.
- Fabric policies reflect the process through which members agree to approve or reject changes to the network, a route, or a smart contract. Policies are agreed upon by channel members when the channel is first set up, but they can also be changed as the channel evolves.

6. Interoperation:

- Allows separate blockchain instances to communicate with one another.
- Interoperability, supported by comprehensive data and transaction standards, is required to capitalize on this powerful technology. The food industry, for example, has made tremendous progress in leveraging data standards to promote food safety and product visibility use cases.
- Interoperability and integration are currently top-of-mind challenges in the blockchain sector.

```
C:\Users\heram>wsl --version
WSL version: 3.0.1.0
Kernel version: 6.18.40.1-1
WSLg version: 1.0.79
MSRDC version: 1.2.7214
Direct3D version: 1.611.1-81528511
DXCore version: 10.0.26100.1-240331-1435.ge-release
Windows version: 10.0.26200.9457

C:\Users\heram>wsl -l -v
Windows Subsystem for Linux has no installed distributions.
You can resolve this by installing a distribution with the instructions
below:

Use 'wsl.exe --list --online' to list available distributions
and 'wsl.exe --install <Distro>' to install.

C:\Users\heram>wsl.exe --install ubuntu
Downloading: Ubuntu
[=====                24.3%                ]
```

Fig 1.1

```
Create a default Unix user account: Ved
Invalid username. A valid username must start with a lowercase letter or
underscore, and can contain lowercase letters, digits, underscores, and
dashes.
Create a default Unix user account: ved
New password:
Retype new password:
passwd: password updated successfully
usermod: no changes
Help improve Ubuntu!
```

Fig 1.2

```
ved@LAPTOP-ENAG2HIV:~$ docker ps
permission denied while trying to connect to the docker API at unix:///v
ar/run/docker.sock
ved@LAPTOP-ENAG2HIV:~$ sudo usermod -aG docker $USER
[sudo: authenticate] Password:
ved@LAPTOP-ENAG2HIV:~$ newgrp docker
Command 'newgrp' not found, but can be installed with:
sudo apt install util-linux-extra
ved@LAPTOP-ENAG2HIV:~$ sudo apt install util-linux-extra
Installing:
  util-linux-extra
Installing dependencies:
  liblastlog2-2
Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 373 kB
  Space needed: 1405 kB / 1025 GB available
Continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 liblastlog2-2
amd64 2.41.3-3ubuntu2 [26.7 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute/main amd64 util-linux-ex
tra amd64 2.41.3-3ubuntu2 [346 kB]
Fetched 373 kB in 2s (200 kB/s)
Selecting previously unselected package liblastlog2-2:amd64.
(Reading database ... 35908 files and directories currently installed.)
Preparing to unpack .../liblastlog2-2_2.41.3-3ubuntu2_amd64.deb ...
Unpacking liblastlog2-2:amd64 (2.41.3-3ubuntu2) ...
Selecting previously unselected package util-linux-extra.
Preparing to unpack .../util-linux-extra_2.41.3-3ubuntu2_amd64.deb ...
Unpacking util-linux-extra (2.41.3-3ubuntu2) ...
Setting up liblastlog2-2:amd64 (2.41.3-3ubuntu2) ...
Setting up util-linux-extra (2.41.3-3ubuntu2) ...
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2.3) ...
ved@LAPTOP-ENAG2HIV:~$
```

Fig 1.3

```
ved@LAPTOP-ENAG2HIV:~$ sudo apt install jq curl git -y
curl is already the newest version (8.18.0-1ubuntu2.4).
curl set to manually installed.
git is already the newest version (1:2.53.0-1ubuntu1).
git set to manually installed.
Installing:
  jq
Installing dependencies:
  libjq1 libonig5
Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 0
  Download size: 393 kB
  Space needed: 1179 kB / 1025 GB available
```

Fig 2.1

```
ved@LAPTOP-ENAG2HIV:~$ curl -fsSL https://deb.nodesource.com/setup_20.x
| sudo -E bash -
sudo: preserving the entire environment is not supported, '-E' is ignored
2026-10-08 03:47:20 - Installing pre-requisites
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [598 kB]
```

Fig 2.2

```
ved@LAPTOP-ENAG2HIV:~$ sudo apt install nodejs -y
Installing:
  nodejs

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 93
  Download size: 32.2 MB
  Space needed: 196 MB / 1024 GB available
```

Fig 2.3

```
ved@LAPTOP-ENAG2HIV:~$ git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 16058, done.
remote: Counting objects: 100% (2970/2970), done.
remote: Compressing objects: 100% (567/567), done.
remote: Total 16058 (delta 2593), reused 2403 (delta 2403), pack-reused 13088 (from 2)
Receiving objects: 100% (16058/16058), 24.09 MiB | 1.25 MiB/s, done.
Resolving deltas: 100% (9241/9241), done.
ved@LAPTOP-ENAG2HIV:~/fabric-samples$
```

Fig 3.1

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.16 does not exist, defaulting to main. fabric-sample
s main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.16 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/downlo
ad/v2.5.16/hyperledger-fabric-linux-amd64-2.5.16.tar.gz
==> Will unpack to: /home/ved/fabric-samples
% Total    % Received % Xferd Average Speed   Time    Time     Time  C
urrent                                 Dload  Upload   Total   Spent    Left   S
peed
  0     0    0     0    0     0     0      0      0      0      0
  0     0    0     0    0     0     0      0      0      0      0
  0     0    0     0    0     0     0      0      0      0      0
121.8M  1 2.13M    0     0  1.37M    0 01:28   00:01   01:27
3121.8M  3 4.27M    0     0  1.66M    0 01:13   00:02   01:11
5121.8M  5 6.10M    0     0  1.69M    0 01:11   00:03   01:08
5121.8M  5 7.26M    0     0  1.57M    0 01:17   00:04   01:13
6121.8M  6 7.82M    0     0  1.38M    0 01:27   00:05   01:22
6121.8M  6 8.13M    0     0  1.21M    0 01:40   00:06   01:34
7121.8M  7 9.18M    0     0  1.19M    0 01:42   00:07   01:35 9
8121.8M  8 10.06M    0     0  1.15M    0 01:45   00:08   01:37 7
8121.8M  8 10.95M    0     0  1.12M    0 01:48   00:09   01:39 7

```

Fig 3.2

```

Status: Downloaded newer image for ghcr.io/hyperledger/fabric-orderer:2.
5.16
ghcr.io/hyperledger/fabric-orderer:2.5.16
====> ghcr.io/hyperledger/fabric-ccenv:2.5.16
2.5.16: Pulling from hyperledger/fabric-ccenv
91094e494159: Pull complete
1a3ecb2ae779: Pull complete
385fa033ccf1: Pull complete
d997db9b0914: Pull complete
c32d94060c0f: Pull complete
1f467a066682: Pull complete
d838ef601b3c: Download complete
Digest: sha256:807493f095f65140ad4cd4543a824c9021a848c8c7aace9bec512e854
5e870f1
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-ccenv:2.5.
16
ghcr.io/hyperledger/fabric-ccenv:2.5.16
====> ghcr.io/hyperledger/fabric-baseos:2.5.16
2.5.16: Pulling from hyperledger/fabric-baseos
9ba18f369df: Pull complete
a151e9a138c: Pull complete
b52a0845a0f4: Pull complete
cbe99fed15ad: Download complete
Digest: sha256:109b4a15b28235db0dcf6ecaa7ffd41dbeeecee6551b231d8178dcf57
4a9157b
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-baseos:2.5
.16
ghcr.io/hyperledger/fabric-baseos:2.5.16
====> Pulling fabric ca Image
====> ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
18afb8c8d2a4: Downloading 16.78MB/43.49MB
b9649d3f054a: Downloading 18.87MB/49.19MB
0b966e606d9a: Download complete
6f4ebca3e823: Downloading 17.83MB/29.54MB
c36bad85367: Download complete

```

Fig 3.3

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples$ docker images | grep fabric
ghcr.io/hyperledger/fabric-baseos:2.5.16      109b4a15b282      257MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB
ghcr.io/hyperledger/fabric-ccenv:2.5.16       807493f095f6      1.01GB
ghcr.io/hyperledger/fabric-orderer:2.5.16     1d918fc7e1e0      182MB
ghcr.io/hyperledger/fabric-peer:2.5.16        58979b65744b      232MB
hyperledger/fabric-baseos:2.5                  109b4a15b282      257MB

```

Fig 3.4

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples$ cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and C
LI delay of '3' seconds and using database 'leveldb with crypto from 'cr
yptogen'
Bringing up network
LOCAL_VERSION=v2.5.16
DOCKER_IMAGE_VERSION=v2.5.16
/home/ved/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-or
g1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-or
g2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-or
derer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
✓Network fabric_test Created 0.1s
✓Volume compose_peer0.org2.example.com Created 0.0s
✓Volume compose_orderer.example.com Created 0.0s
✓Volume compose_peer0.org1.example.com Created 0.0s
✓Container peer0.org2.example.com Started 0.8s
✓Container peer0.org1.example.com Started 0.6s
✓Container orderer.example.com Started 1.1s
CONTAINER ID IMAGE COMMAND C
REATED STATUS PORTS

```

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ ./network.sh deployCC
-ccn basic -ccp ../asset-transfer-basic/chaincode-javascript -ccl javas
cript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer
-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep 'basic.1.0:1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15c
bcc7f59b629$'
+ peer lifecycle chaincode queryinstalled --output json
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz

```

```

2026-10-08 04:14:01.323 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid
[b8dc37b58e486ac71c33739abf2a18af7c241b59e4b58df61708df1344858b6] commi
tted with status (VALID) at localhost:7051
2026-10-08 04:14:01.323 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid
[b8dc37b58e486ac71c33739abf2a18af7c241b59e4b58df61708df1344858b6] commi
tted with status (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 second
s.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name b
asic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychann
el':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin:
vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychanne
l'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 second
s.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name b
asic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychann
el':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin:
vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychanne
l'
Chaincode initialization is not required
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$

```

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ export FABRIC_CFG_PAT
H=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/org
anizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.c
om/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organiz
ations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/m
sp
export CORE_PEER_ADDRESS=localhost:7051
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tl
s --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peer0
rganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --
peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/tes
t-network/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt -c '{"function":"InitLedger","Args":[]}'
2026-10-08 04:16:42.921 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$

```

Fig 4.1

Fig 4.2

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode query
-C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Si
ze":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset
2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Col
or":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},
{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Siz
e":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"ass
et5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":80
0,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"as
set"}]
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode query
-C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","S
ize":15,"docType":"asset"}
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$

```

Fig 4.3

```

ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tl
s --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peer0
rganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --
peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/tes
t-network/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt -c '{"function":"CreateAsset","Args":["asset7
","blue","50","Manav","100"]}'
2026-10-08 04:19:43.048 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":
\\\"asset7\\\",\\\"Color\\\":\\\"blue\\\",\\\"Size\\\":50,\\\"Owner\\\":\\\"Manav\\\",\\\"Appraise
dValue\\\":100}"
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ peer chaincode invoke
-o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tl
s --cafile $HOME/fabric-samples/test-network/organizations/ordererOrgani
zations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.ex
ample.com-cert.pem -C mychannel -n basic --peerAddresses localhost:7051
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peer0
rganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --
peerAddresses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/tes
t-network/organizations/peerOrganizations/org2.example.com/peers/peer0.o
rg2.example.com/tls/ca.crt -c '{"function":"TransferAsset","Args":["asse
t7","Alice"]}'
2026-10-08 04:19:53.640 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQu
ery -> Chaincode invoke successful. result: status:200 payload:"Manav"
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$

```

Fig 4.4

```

2026-10-08 04:21:22.137 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Alice"
ved@LAPTOP-ENAG2HIV:~/fabric-samples/test-network$ cd ~/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org2.example.com      Removed      1.0s
✓Container peer0.org1.example.com      Removed      0.9s
✓Container orderer.example.com         Removed      0.4s
✓Volume compose_orderer.example.com    Removed      0.0s
✓Volume compose_peer0.org2.example.com Removed      0.0s
✓Network fabric_test                   Removed      0.2s
✓Volume compose_peer0.org1.example.com Removed      0.0s

```

Fig 5.0

Conclusion: Successfully implementing a Hyperledger Fabric network, deploying chaincode, and managing assets on WSL

Commands used

Install dependencies: `cd ~`

`sudo apt update && sudo apt upgrade -y`

`sudo apt install jq curl git -y`

`curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -`

`sudo apt install nodejs -y`

Clone Repo and hyperledgerfabric installation: `git clone`

`https://github.com/hyperledger/fabric-samples`

`cd fabric-samples`

`curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh`

`chmod +x install-fabric.sh`

`./install-fabric.sh docker samples binary`

`docker images | grep fabric`

Path initialization: `export PATH=$PATH:$HOME/fabric-samples/bin`

`export FABRIC_CFG_PATH=$HOME/fabric-samples/config`

`cd ~/fabric-samples/test-network`

`./network.sh up createChannel`

Deploy Chaincode: `./network.sh deployCC -ccn basic -ccp`

`../asset-transfer-basic/chaincode-javascript -ccl javascript`

Set peer network: `export FABRIC_CFG_PATH=$HOME/fabric-samples/config`

`export PATH=$PATH:$HOME/fabric-samples/bin`

`export CORE_PEER_TLS_ENABLED=true`

`export CORE_PEER_LOCALMSPID="Org1MSP"`

`export`

`CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrg`

`anizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt`

`export`

`CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganiza`

`tions/org1.example.com/users/Admin@org1.example.com/msp`

`export CORE_PEER_ADDRESS=localhost:7051`

Initialize Ledger: `peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride`

`orderer.example.com --tls --cafile`

`$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/order`

`er.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C mychannel -n basic --peerAddresses`

```

localhost:7051 --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.
org1.example.com/tls/ca.crt --peerAddresses localhost:9051 --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.
org2.example.com/tls/ca.crt -c '{"function":"InitLedger","Args":[]}'
Query: peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
Read,Create,Transfer Assets: peer chaincode query -C mychannel -n basic -c
'{"Args":["ReadAsset","asset5"]}'
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls
--cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/order
er.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C mychannel -n basic --peerAddresses
localhost:7051 --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.
org1.example.com/tls/ca.crt --peerAddresses localhost:9051 --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.
org2.example.com/tls/ca.crt -c
'{"function":"CreateAsset","Args":["asset7","blue","50","Manav","100"]}'

peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls
--cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/order
er.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C mychannel -n basic --peerAddresses
localhost:7051 --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.
org1.example.com/tls/ca.crt --peerAddresses localhost:9051 --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.
org2.example.com/tls/ca.crt -c '{"function":"TransferAsset","Args":["asset7","Alice"]}'

```